

DevOps

DevOps es un término popular común en el desarrollo de software. Pero tanto si eres nuevo en el sector como si llevas años en él, debes saber qué es DevOps y cómo puede afectar a tu forma de trabajar. En esta lectura, explorarás DevOps y analizarás su cultura y sus componentes comunes.

¿Qué es DevOps?

La definición de DevOps depende de a quién le preguntes. Pero, para la mayoría de los expertos, se basa en lo siguiente:

***DevOps** es un enfoque de desarrollo de software en el que los equipos de desarrollo (Dev) y operaciones (Ops) colaboran para ofrecer actualizaciones de software rápidas y de alta calidad.*

Desglosemos los componentes clave de esta definición.

Desarrollo

El componente «Dev» de DevOps se refiere al equipo de **desarrollo**. Este equipo escribe, edita y documenta el código del software y lo mantiene en una base de código o repositorio.

Operaciones de TI

Por otro lado, el elemento «Ops» alude al **equipo de operaciones de tecnología de la información (TI)**. Este equipo configura, gestiona y optimiza la infraestructura de la que depende el desarrollo, incluidas las actualizaciones de hardware, software, red y almacenamiento en la nube. Asimismo, el equipo implementa actualizaciones de software para usuarios y respalda el código.

Velocidad

Uno de los objetivos de DevOps consiste en lanzar los productos o servicios que forman parte de un proyecto de forma rápida y frecuente mediante una implementación ágil. Con DevOps, las organizaciones publican actualizaciones pequeñas pero continuas, en lugar de cambios grandes y poco frecuentes.

Calidad

Los entregables deben ser de alta calidad, lo que significa que cumplen con las expectativas de las partes interesadas en cuanto a rendimiento, funcionalidad, estabilidad, fiabilidad, disponibilidad y seguridad. Dichas partes interesadas incluyen no solo equipos de desarrollo y operaciones, sino también equipos de seguridad, equipos de línea de negocio y usuarios, entre otros.

Cultura

DevOps solo funciona si la organización adopta una cultura DevOps, cuya piedra angular es la **colaboración**.

Tradicionalmente, los equipos de desarrollo y operaciones han trabajado por separado en sus propios silos. Sin embargo, con DevOps trabajan codo con codo y mantienen una comunicación abierta durante todo el ciclo de vida del **desarrollo de software (SDLC)**. Comparten información con libertad y transparencia y colaboran para garantizar que los desarrolladores reciban **comentarios rápidos** sobre cualquier problema de software.

Además, los equipos también **comparten la responsabilidad** de la calidad del producto.

- Los desarrolladores no solo crean el código, sino que lo prueban y ayudan a mantenerlo a lo largo del tiempo, en lugar de dejar estas tareas al equipo de operaciones.
- Por su parte, el personal de operaciones participa en la planificación de discusiones para actualizaciones de software. De este modo, se aseguran de que los desarrolladores incorporen la supervisión, el registro, las pruebas de seguridad y otras funciones que requiere el sistema.

Al colaborar a lo largo del SDLC, los equipos comprenden mejor las necesidades de los demás y, a su vez, pueden ajustar sus prácticas de trabajo para responder a esas necesidades, mejorando la compenetración del equipo y eliminando los obstáculos que retrasan los lanzamientos.

Componentes

Analicemos otros componentes comúnmente reconocidos de DevOps.

Agilidad

DevOps normalmente implica un enfoque ágil del desarrollo de software. Para explicarlo, hablemos primero de su predecesor, el modelo en cascada.

El **modelo en cascada** es lineal: el equipo sigue un plan largo y detallado con fases secuenciales diferenciadas, como la planificación, el diseño, el desarrollo, las pruebas y el soporte. Este plan especifica unos plazos para cada fase y unas responsabilidades para cada miembro del equipo, y el objetivo consiste en ejecutar el plan con la máxima precisión posible.

No obstante, este método presenta varios problemas para el desarrollo de software.

- A medida que el proyecto avanza, los desarrolladores pueden encontrar obstáculos o pensar en nuevas y mejores ideas, y los clientes pueden cambiar sus requisitos. Sin embargo, el planteamiento inflexible hace que realizar adaptaciones sea difícil y, en ocasiones, costoso.
- Y lo que es peor: los clientes y usuarios normalmente no ven el producto hasta la fase de prueba, por lo que los desarrolladores no reciben comentarios hasta ese momento. Abordar los errores y los problemas de los clientes en una fase tan tardía del proceso puede resultar más costoso y requerir más tiempo que en las primeras fases de desarrollo.

Por el contrario, el **desarrollo ágil** es iterativo. En lugar de completar el proyecto en fases más largas y diferenciadas, los equipos trabajan en iteraciones breves, denominadas «sprints», que suelen durar varias semanas. El objetivo de cada sprint es producir una pequeña entrega para que el cliente pueda comunicar sus comentarios rápidamente, lo que permite a los desarrolladores conocer los problemas del cliente en una fase temprana y resolverlos a medida que se avanza. Además, se llevan a cabo pruebas en cada sprint, en lugar de hacerlo al final del proyecto, con el fin de que los equipos identifiquen y solucionen los errores antes.

Al igual que un proyecto en cascada, un proyecto ágil también tiene un objetivo general. Sin embargo, el plan es menos detallado y más flexible para adaptarse a nuevas ideas y requisitos.

En resumen, el método ágil es ideal para DevOps.

- Centrarse en sprints breves, entregables pequeños y pruebas continuas encaja bien con el objetivo de DevOps de producir actualizaciones de software frecuentes y de calidad.
- Los desarrolladores y el personal de operaciones pueden adaptarse rápidamente a los cambios en las exigencias y estándares de calidad de los proyectos.

Microservicios

Para acelerar la implementación, DevOps requiere un diseño de aplicaciones centrado en los microservicios. Un **microservicio** es un componente pequeño de una aplicación que funciona de forma independiente y que puede funcionar junto con otros componentes para componer la aplicación completa.

En el desarrollo tradicional, los desarrolladores construyen todas las partes de una aplicación en una pieza grande y monolítica. Por este motivo, al agregar una función o corregir un error, los desarrolladores deben volver a implementar toda la aplicación.

Sin embargo, al utilizar microservicios, los desarrolladores solo necesitan editar el componente relevante, en lugar de la aplicación al completo. Estos cambios a menor escala hacen que las actualizaciones sean más rápidas y reducen el riesgo de dañar la aplicación.

Automatización

DevOps implica el uso de herramientas para **automatizar** el movimiento del código a través del SDLC.

Tomemos como ejemplo las pruebas. El código debe someterse a distintos niveles de pruebas a lo largo de su ciclo de vida, y hay diversas herramientas que pueden ejecutar automáticamente las pruebas adecuadas para cada etapa. Las herramientas también pueden preparar y configurar los componentes de la infraestructura, controlar el rendimiento y los problemas de seguridad, identificar componentes o activos incompatibles e implementar el código en producción.

La automatización reduce el tiempo de implementación y la posibilidad de que se produzcan errores humanos. También libera a los desarrolladores y al personal de operaciones para que se centren en la codificación, la resolución de problemas y otras tareas cruciales para mejorar el producto.

Integración continua y entrega continua

La integración continua y la entrega continua (CI/CD) son dos procesos automatizados esenciales para DevOps.

- La **integración continua (CI)** es un proceso en el que los desarrolladores integran periódicamente pequeñas piezas de código nuevo en su base de código principal. El código se somete a pruebas automáticas y el administrador de la base de código lo revisa y aprueba antes de su integración.

- A continuación viene la **entrega continua (CD)**. Se trata de un proceso que envía este nuevo código a un entorno de preproducción para verificar que esté listo para una implementación rápida y segura en producción. En esta fase, el código se somete a pruebas y revisiones adicionales y posteriormente pasa a producción.

CI/CD es un concepto de gran valor para DevOps.

- Automatiza las pruebas, lo que ahorra tiempo que de otro modo debería dedicarse a probar el código de forma manual y proporciona comentarios instantáneos a los desarrolladores sobre los errores.
- Automatiza las comprobaciones de ciberseguridad, como el análisis de aplicaciones y vulnerabilidades, que forman parte de **DevSecOps** (DevOps con seguridad integrada).
- Las pruebas, el escaneo y la revisión frecuentes también se traducen en un código de mayor calidad.
- CI/CD automatiza la implementación en el entorno de preproducción.

Con CI/CD, las actualizaciones llegan a producción más rápido y con menos esfuerzo.